

PostgreSQL Interview Quick Reference

One-page answers for the 30 most common PostgreSQL interview questions. Designed for last-minute review before an interview.

Fundamentals

Q1. What is MVCC and how does it work in PostgreSQL?

MVCC (Multi-Version Concurrency Control) allows readers and writers to not block each other. Each row version stores `xmin` (transaction that created it) and `xmax` (transaction that deleted/updated it). A transaction sees a row if `xmin` is committed and `xmax` is zero, aborted, or in the future relative to the transaction's snapshot. Updates create a new row version rather than modifying in place.

Key benefit: Readers never block writers; writers never block readers. **Key cost:** Dead row versions accumulate → autovacuum must clean them up.

Q2. Explain the difference between DELETE, TRUNCATE, and DROP.

Command	Removes	Logs	Transaction-Safe	Resets Seq	Cascade
DELETE	Rows (with WHERE)	Row-by-row (WAL)	Yes	No	Via FK cascade
TRUNCATE	All rows	Table-level	Yes (in PG)	Optional	With CASCADE
DROP	Table + data + indexes + constraints	DDL	Yes	N/A	With CASCADE

Q3. What is WAL and why does it exist?

WAL (Write-Ahead Log) is a sequential log of all database changes. Before any data file is modified, the change is written to WAL first. This enables:

- **Crash recovery:** replay WAL to reach consistent state
- **Point-in-Time Recovery (PITR):** replay WAL to any past timestamp
- **Streaming replication:** stream WAL to standbys
- **Logical replication:** decode WAL to replicate specific tables/rows

Q4. What is a vacuum and when is it needed?

VACUUM reclaims space from dead row versions (created by UPDATE/DELETE under MVCC). Without it:

- Table and index **bloat** grows, slowing queries
- Statistics become stale (planner makes bad choices)
- **XID wraparound** occurs after ~2 billion transactions — catastrophic

`VACUUM` marks dead space as reusable. `VACUUM FULL` rewrites the table (locks it). `AUTOVACUUM` runs automatically. Never disable it.

Q5. What is the difference between READ COMMITTED and REPEATABLE READ?

	READ COMMITTED (default)	REPEATABLE READ
Snapshot taken	Per statement	Per transaction
Non-repeatable reads	Possible	Prevented
Phantom reads	Possible	Prevented (PG extension)
Use case	Most OLTP queries	Reports, multi-step read consistency

Indexes

Q6. Explain the difference between B-tree, GIN, and GiST indexes.

Index	Best For	Operators
B-tree (default)	Scalars: equality + range	=, <, >, BETWEEN, LIKE 'prefix%'
GIN	Multi-valued: arrays, JSONB, FTS	@>, ?, @@, element-level
GiST	Geometry, ranges, FTS with ranking	&&, @>, <@, <-> (kNN)

Q7. When would you use a partial index?

When most queries filter on a specific condition. Example: `WHERE status = 'active'` (only 5% of rows). A partial index only indexes those rows — smaller, faster to build, faster to scan.

```
CREATE INDEX idx_active_users ON users (email) WHERE is_active = TRUE;
```

Savings: 95% smaller index; 95% fewer index entries to maintain on writes.

Q8. What is a covering index?

An index that contains all columns needed by a query — no heap access required. PostgreSQL calls this an "Index Only Scan."

```
CREATE INDEX idx_covering ON orders (customer_id, status) INCLUDE (total_amount);
-- Query: SELECT total_amount WHERE customer_id=X AND status='Y'
-- → Index Only Scan (no heap fetch needed)
```

Q9. How does PostgreSQL decide whether to use an index or a seq scan?

The planner compares costs:

- **Seq scan:** `seq_page_cost × pages`
- **Index scan:** `random_page_cost × index_pages + cpu costs`

If selectivity is low (many rows match), seq scan wins (sequential I/O is faster than many random I/Os). Key settings: `random_page_cost` (set to 1.1 for SSD), `effective_cache_size`, row count statistics.

Q10. What is index bloat and how do you fix it?

After many UPDATES/DELETES, index pages contain many dead entries. The index stays large but has sparse useful data. Fix:

```
REINDEX INDEX CONCURRENTLY idx_name; -- non-blocking rebuild
-- or
VACUUM table_name; -- reclaims dead tuple references in indexes
```

Query Optimization

Q11. Explain the three join algorithms PostgreSQL uses.

Algorithm	How It Works	Best For
Nested Loop	For each outer row, scan inner	Outer is small; inner has index on join key
Hash Join	Build hash table of inner; probe with outer	Medium/large tables, no useful index
Merge Join	Merge two pre-sorted streams	Both sides sorted or sortable efficiently

Q12. What is a correlated subquery and why is it slow?

A correlated subquery references columns from the outer query and re-executes for each outer row — $O(n^2)$.

```
-- Slow: executes subquery for every row
SELECT e.name, (SELECT AVG(salary) FROM employees e2 WHERE e2.dept_id = e.dept_id)
FROM employees e;

-- Fast: window function (single pass)
SELECT name, AVG(salary) OVER (PARTITION BY dept_id) FROM employees;
```

Q13. When is a CTE materialized vs. inlined?

PostgreSQL 12+:

- Default: CTEs are **inlined** (treated as subqueries, planner can optimize through them)
- Use `WITH cte AS MATERIALIZED (...)` to force evaluation as a separate step
- Pre-PG12: ALL CTEs were materialized (optimization fence)

Force materialization when: CTE result is referenced multiple times (avoids re-evaluation).

Q14. How do you find slow queries in PostgreSQL?

```
-- pg_stat_statements (must enable extension)
SELECT query, calls, round(total_exec_time/calls::numeric,2) AS avg_ms
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;

-- Currently running queries
SELECT pid, now()-query_start AS duration, query
FROM pg_stat_activity WHERE state='active' ORDER BY duration DESC;
```

```
-- Slow query log
SET log_min_duration_statement = 1000; -- log queries > 1s
```

Q15. How do you interpret EXPLAIN ANALYZE output?

Key fields:

- `cost=X..Y` : planner's estimated cost (X=startup, Y=total)
- `rows=N` : estimated rows
- `actual time=X..Y` : real ms (startup..total)
- `actual rows=N` : real rows
- `loops=N` : times this node executed (multiply actual time by loops)
- `Buffers: shared hit/read` : cache hits vs. disk reads

Red flags: large gap between `rows` estimated and actual; `Sort Method: external merge` (work_mem spill); `Rows Removed by Filter: NNNN` (wasteful scan).

Schema Design

Q16. What are the normal forms and when do you denormalize?

Form	Rule
1NF	Atomic columns, no repeating groups
2NF	1NF + no partial dependencies on composite PK
3NF	2NF + no transitive dependencies
BCNF	Stronger 3NF — every determinant is a candidate key

Denormalize when: Read performance is critical and join cost is unacceptable. Cache frequently computed values. Trade write complexity for read speed. Always benchmark first.

Q17. Explain table partitioning in PostgreSQL.

Partitioning splits a large table into smaller physical partitions, allowing:

- **Partition pruning:** only scan relevant partitions
- **Easier archival:** `DROP PARTITION` instead of slow `DELETE`
- **Parallel query:** scan multiple partitions simultaneously

```
-- Range partitioning by date
CREATE TABLE events (...) PARTITION BY RANGE (occurred_at);
CREATE TABLE events_2024_01 PARTITION OF events
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
```

Types: RANGE, LIST, HASH. Default partition catches everything else.

Q18. What is a materialized view and when do you use it?

A materialized view stores the query result physically (like a table). Unlike a regular view, it doesn't recompute on every query.

```
CREATE MATERIALIZED VIEW daily_stats AS SELECT ... GROUP BY ...;
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_stats; -- non-blocking refresh
CREATE UNIQUE INDEX ON daily_stats (day); -- required for CONCURRENTLY
```

Use when: expensive aggregation needed frequently; data can be slightly stale; base query takes > 1 second.

Q19. What is the difference between JSONB and JSON in PostgreSQL?

	JSON	JSONB
Storage	Text (preserves whitespace, order)	Binary (parsed, optimized)
Indexing	Not indexable directly	GIN, B-tree on extracted fields
Query speed	Slower (re-parses)	Faster
Write speed	Faster	Slightly slower (parse + store)
Recommended	Rarely	Almost always

Q20. How does PostgreSQL handle NULL values?

- `NULL = NULL` evaluates to `NULL` (not TRUE) — use `IS NULL`
- `NOT IN` (subquery with NULLs) returns NO ROWS — use `NOT EXISTS`
- NULL is ignored by aggregate functions (`COUNT(*)` counts NULLs; `COUNT(col)` does not)
- `COALESCE(col, default)` returns first non-NULL
- `NULLIF(a, b)` returns NULL if a=b (used to avoid division by zero: `NULLIF(denominator, 0)`)

Replication & High Availability

Q21. Explain streaming replication in PostgreSQL.

Primary continuously streams WAL records to standby(s) via TCP. Standby applies WAL to stay in sync. Standby can serve read queries. Key parameters:

- `wal_level = replica` (minimum for replication)
- `synchronous_standby_names` (for synchronous replication)
- Check lag: `SELECT now() - pg_last_xact_replay_timestamp() AS lag;`

Sync vs. async: Synchronous = primary waits for standby to confirm before commit (zero data loss, latency hit). Async = primary doesn't wait (possible data loss on failover, no latency hit).

Q22. What is logical replication and when is it used?

Logical replication replicates data changes (not physical WAL blocks) based on row changes. Enables:

- Replicate specific tables (not entire cluster)
- Replicate to different PostgreSQL versions
- Replicate to other databases (CDC via Debezium, etc.)
- Zero-downtime major version upgrades

```
-- Publisher (primary)
CREATE PUBLICATION my_pub FOR TABLE orders, customers;
-- Subscriber (replica)
CREATE SUBSCRIPTION my_sub CONNECTION '...' PUBLICATION my_pub;
```

Advanced Features

Q23. What is row-level security (RLS) and how does it work?

RLS allows defining policies that restrict which rows a user can see or modify. Each query automatically has the policy appended as an invisible WHERE clause.

```
ALTER TABLE tenant_data ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON tenant_data
    USING (tenant_id = current_setting('app.tenant_id')::bigint);
-- Every SELECT/INSERT/UPDATE/DELETE only sees rows for current tenant
```

Q24. How does PostgreSQL's planner use statistics?

`pg_statistic` stores per-column statistics collected by ANALYZE:

- Most common values (MCV) and their frequencies
- Histogram buckets for range estimation
- NULL fraction
- Distinct value count

Planner uses these to estimate selectivity of WHERE conditions → estimate rows → choose optimal plan. Poor estimates → bad plans → slow queries.

Q25. What are advisory locks and when would you use them?

Application-defined locks that PostgreSQL manages but doesn't associate with any database object. Use for: ensuring one process runs at a time, distributed coordination, custom serialization.

```
SELECT pg_advisory_xact_lock(hashtext('sync_job_products')); -- in transaction
SELECT pg_try_advisory_lock(42); -- non-blocking (returns boolean)
```

Transactions & Locking

Q26. What causes deadlocks and how do you prevent them?

Deadlock: Transaction A holds lock X and waits for Y; Transaction B holds Y and waits for X. PostgreSQL detects and aborts one automatically.

Prevention:

1. Always acquire multiple locks in the same order across all transactions
2. Keep transactions short
3. Use `SELECT FOR UPDATE SKIP LOCKED` for queue patterns
4. Use `LOCK TABLE` explicitly at start if locking multiple tables

Q27. Explain SELECT FOR UPDATE vs SELECT FOR SHARE.

Command	Lock Mode	Use Case
SELECT FOR UPDATE	Exclusive row lock	Read then update; prevent others from modifying
SELECT FOR SHARE	Share row lock	Read then check FK; prevent others from deleting
SELECT FOR UPDATE SKIP LOCKED	Skip locked rows	Job queue: take next available job
SELECT FOR UPDATE NOWAIT	Fail immediately if locked	Non-blocking lock attempt

Operations

Q28. How do you do a zero-downtime schema migration?

Avoid `ALTER TABLE` with `ACCESS EXCLUSIVE` lock on large tables. Use patterns:

1. **Add nullable column:** `ALTER TABLE t ADD COLUMN new_col TYPE DEFAULT NULL;` — fast
2. **Add column with default (PG 11+):** Stored default is metadata-only (instant)
3. **Backfill in batches:** `UPDATE t SET new_col = val WHERE id BETWEEN X AND Y;`
4. **Create index CONCURRENTLY:** doesn't block reads/writes
5. **Add constraint as NOT VALID then VALIDATE:** splits lock window

```
ALTER TABLE t ADD CONSTRAINT fk_x FOREIGN KEY (col) REFERENCES other(id) NOT VALID;
ALTER TABLE t VALIDATE CONSTRAINT fk_x; -- shares lock, can be slower
```

Q29. What is connection pooling and why is it needed?

Each PostgreSQL connection is an OS process (~5–10MB RAM, startup overhead ~30ms). Applications may create thousands of short-lived connections, overwhelming PostgreSQL.

PgBouncer sits between app and PostgreSQL:

- Maintains a small pool of actual PostgreSQL connections
- Hands them out to app connections on-demand
- Modes: session, transaction (recommended), statement

Result: 5,000 app connections → 100 PostgreSQL connections. PostgreSQL stays healthy.

Q30. What is XID wraparound and how is it prevented?

Transaction IDs (XIDs) are 32-bit integers. After ~2.1 billion transactions, they wrap around. Rows with future XIDs become invisible → **data loss**. PostgreSQL uses aggressive autovacuum to freeze old XIDs before this happens.

```
-- Check age of oldest unfrozen XID
SELECT datname, age(datfrozenxid) FROM pg_database;
```

```
-- Age > 1.5 billion → take action immediately!

-- Force freeze
VACUUM FREEZE table_name;
```

Prevention: Healthy autovacuum + monitoring XID age. Alert at 500M age, emergency at 1.5B.

Quick Cheat Card — Pre-Interview 5-Minute Review

1. MVCC = row versions, readers don't block writers
2. WAL = crash recovery + replication
3. VACUUM = clean dead rows, update stats, prevent XID wraparound
4. Isolation default = READ COMMITTED (snapshot per statement)
5. B-tree = most indexes; GIN = arrays/JSONB/FTS; GiST = geo/ranges
6. random_page_cost = 1.1 for SSD (critical!)
7. EXPLAIN ANALYZE = always use to debug; never trust estimates alone
8. CTE = inlined by default in PG12+; use MATERIALIZED to force cache
9. NOT IN with NULLs = returns empty; use NOT EXISTS instead
10. Partitioning = RANGE for dates, HASH for even distribution, LIST for enum
11. Materialized view = precomputed table; REFRESH CONCURRENTLY for live
12. Row-level security = automatic WHERE clause per policy
13. SKIP LOCKED = job queue pattern; NOWAIT = fail if locked
14. Advisory locks = app-level coordination without DB objects
15. PgBouncer = essential for production (transaction pooling mode)
16. shared_buffers = 25% RAM; work_mem = per-sort, not total
17. autovacuum = never disable; monitor n_dead_tup and xid age
18. Covering index + INCLUDE = index-only scan (fastest reads)
19. JSONB > JSON for almost everything
20. Deadlock prevention = consistent lock ordering across transactions